

Calibrating the Implied Volatility Surface of NIFTY Options Using Polynomial Regression and Market Data

1 Objective:

To calibrate a parametric model for the implied volatility (IV) surface of NIFTY index options using real market data, enabling accurate IV interpolation for arbitrary strikes and maturities. Key goals included:

- Filtering liquid option data and computing IVs via Black-Scholes and Newton-Raphson.
 - Modeling IV as a function of log-moneyness ($k = \log(K/F)$, where K = strike, F = forward price) and time to maturity (T).
 - Selecting an optimal polynomial degree using cross-validation.
 - Visualizing the 3D IV surface and diagnosing model residuals.
 - Evaluating model performance and identifying areas for improvement.
-

2 Methodology:

2.1 Data Collection & Filtering:

- Source: NSE India option chain data (11 expiries: Jun 2025–Jun 2026).
- Filters: Only call options with bid ask spread $\leq 25\%$ and volume ≥ 8 .
 - Mid-price used as market price: $\text{call_price} = (\text{BID} + \text{ASK})/2$.
- Key Variables:
 - Spot price: 24,750.70
 - Risk-free rate: 6%
 - Valuation date: May 30, 2025
- Final Dataset: 297 options after cleaning.

2.2 Preprocessing:

- Implied volatility computing using Black Scholes model via Newton Raphson.
- Derived forward price (F) and log-moneyness ($k = \log(K/F)$).
- IQR method used to detect outliers of implied volatilities.

2.3 Model Calibration

- Regressed implied volatility on maturity and log-moneyness using polynomial regression (degrees 1–9).
- Employed 5-fold cross-validation with 10 random seeds to ensure robustness.
- Performance measured using RMSE and R^2 .

2.4 Model Selection:

- Degree-2 polynomial regression yielded the best trade-off: $\text{RMSE} \approx 0.006$ and $R^2 \approx 0.59$.

2.5 Visualization & Diagnostics:

- Plotted the 3D implied volatility surface using Plotly.
 - Analyzed residuals: scatter plots, histograms, and Q-Q plots to assess model adequacy.
-

3 Key Findings:

- The implied volatility surface exhibited typical features: skew and term structure.
 - A degree-2 polynomial model captured the main structure with good accuracy and interpretability.
 - Residual diagnostics revealed: Mostly centered residuals with mild curvature and tail deviations. Indications of remaining nonlinearities and asymmetric market effects.
-

4 Code:

Libraries

```
[1]: import numpy as np
import pandas as pd
import scipy.stats as stats

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score, make_scorer

import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objs as go
```

Data Filtering and Preprocessing:

We obtained NIFTY option chain data from NSE India, spanning maturities from June 5, 2025 to June 25, 2026. Only call options were considered. To ensure quality and liquidity, we retained entries with a bid-ask spread $\leq 25\%$ and volume ≥ 8 . The mid-price (average of bid and ask) was used as the call price.

```
[2]: df_list = []

spot_price = 24750.70
risk_free_rate = 0.06

valuation_date = pd.Timestamp('2025-05-30')
expiry_dates = ["05-Jun-2025", "12-Jun-2025", "19-Jun-2025", "26-Jun-2025",
↳ "03-Jul-2025", "31-Jul-2025", "28-Aug-2025", "25-Sep-2025", "24-Dec-2025",
↳ "26-Mar-2026", "25-Jun-2026"]

for dates in expiry_dates:
    file_name = "option-chain-ED-NIFTY-"+dates+".csv"
    tem_df = pd.read_csv(file_name, skiprows= 1)
    imp_cols = ["BID", "ASK", "VOLUME", "STRIKE"]
    tem_df = tem_df[imp_cols]

    for col in imp_cols:
        tem_df[col] = (
            tem_df[col].astype(str).str.replace(",", "", regex=False).str.
↳ strip().replace("-", pd.NA)
        )
        tem_df[col] = pd.to_numeric(tem_df[col], errors='coerce')

    for col in imp_cols:
        tem_df[col].astype(str).str.replace(",", "", regex=False).str.strip()

        tem_df.loc[:,col] = pd.to_numeric(tem_df[col],errors='coerce')

    filtered_df = tem_df[
        (tem_df['BID'] > 0) &
        (tem_df['ASK'] > 0) &
        (tem_df['ASK'] / tem_df['BID'] < 1.25) &
        (tem_df['VOLUME'] > 8)].copy()

    filtered_df["call_price"] = 0.5 * ( filtered_df["BID"] + filtered_df["ASK"])

    filtered_df["maturity"] = (pd.to_datetime(dates, format= "%d-%b-%Y") -
↳ valuation_date).days / 365

    filtered_df = filtered_df.drop(columns= ["BID", "ASK", "VOLUME"])

    df_list.append(filtered_df)
```

```
merged_data = pd.concat(df_list, ignore_index=True)
```

EDA

```
[3]: print("Shape:", merged_data.shape)
      print("\nInfo:", merged_data.info())

      print("\nData Types:\n", merged_data.dtypes)

      print("\nFirst 5 Rows:\n", merged_data.head())

      print("\nMissing Values:\n", merged_data.isnull().sum())

      print("\nSummary Statistics (numerical):\n", merged_data.describe().T)
```

Shape: (297, 3)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 297 entries, 0 to 296

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	STRIKE	297 non-null	float64
1	call_price	297 non-null	float64
2	maturity	297 non-null	float64

dtypes: float64(3)

memory usage: 7.1 KB

Info: None

Data Types:

STRIKE	float64
call_price	float64
maturity	float64
dtype:	object

First 5 Rows:

	STRIKE	call_price	maturity
0	22500.0	2269.400	0.016438
1	22800.0	1972.450	0.016438
2	23000.0	1782.375	0.016438
3	23400.0	1393.225	0.016438
4	23450.0	1327.750	0.016438

Missing Values:

STRIKE	0
call_price	0

```
maturity      0
dtype: int64
```

Summary Statistics (numerical):

	count	mean	std	min	25% \
STRIKE	297.0	25215.319865	1458.059839	19000.000000	24500.000000
call_price	297.0	518.017677	835.880644	1.475000	33.600000
maturity	297.0	0.091499	0.116397	0.016438	0.035616

	50%	75%	max
STRIKE	25200.000000	26050.000000	31000.000000
call_price	230.550000	611.075000	5807.000000
maturity	0.054795	0.073973	0.569863

Black-Scholes Call Price

```
[4]: def black_scholes_call(S, K, T, r, sigma):
      d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
      d2 = d1 - sigma * np.sqrt(T)
      call_price = S * stats.norm.cdf(d1) - K * np.exp(-r * T) * stats.norm.cdf(d2)
      return call_price
```

Implied Volatility via Newton-Raphson

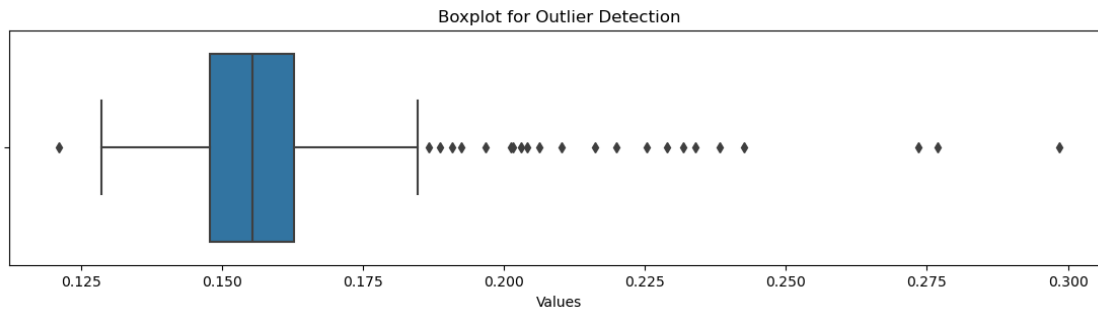
```
[5]: def implied_vol_newton(Call_market, S, K, T, r, tol=1e-6, max_iter=100):
      sigma = 0.2 # initial guess
      for i in range(max_iter):
          if sigma <= 1e-8 or sigma > 5:
              return np.nan
          price = black_scholes_call(S, K, T, r, sigma)
          vega = S * stats.norm.pdf((np.log(S / K) + (r + 0.5 * sigma ** 2) * T) /
      ↪(sigma * np.sqrt(T))) * np.sqrt(T)
          diff = price - Call_market
          if abs(diff) < tol:
              return sigma
          if vega == 0 or np.isnan(vega):
              return np.nan
          sigma -= diff / vega
      return np.nan
```

```
[6]: iv_list = []
      for i in range(len(merged_data)):
          iv = implied_vol_newton(merged_data["call_price"][i], spot_price,
      ↪merged_data["STRIKE"][i], merged_data["maturity"][i], risk_free_rate)
          iv_list.append(iv)

      merged_data["implied_vol"] = iv_list
```

Boxplot for outlier detection.

```
[7]: plt.figure(figsize=(14, 3))
sns.boxplot(x=merged_data["implied_vol"])
plt.title('Boxplot for Outlier Detection')
plt.xlabel('Values')
plt.show()
```



Remove outliers.

```
[8]: Q1 = merged_data['implied_vol'].quantile(0.25)
Q3 = merged_data['implied_vol'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

vol_data = merged_data[
    (merged_data['implied_vol'] >= lower_bound) &
    (merged_data['implied_vol'] <= upper_bound)
].copy()

vol_data.isnull().any()
```

```
[8]: STRIKE          False
call_price          False
maturity            False
implied_vol         False
dtype: bool
```

```
[9]: vol_data = vol_data.copy()
vol_data.loc[:, "forward"] = spot_price * np.exp(risk_free_rate *
    ↳ vol_data["maturity"])
temp1 = vol_data["STRIKE"] / vol_data["forward"]
vol_data.loc[:, "log_moneyness"] = [np.log(x) for x in temp1]
df = vol_data.copy()
```

```
df.head()
```

```
[9]:
```

	STRIKE	call_price	maturity	implied_vol	forward	log_moneyness
6	23600.0	1175.500	0.016438	0.160719	24775.123692	-0.048593
9	23800.0	982.750	0.016438	0.177398	24775.123692	-0.040154
10	23850.0	932.150	0.016438	0.167321	24775.123692	-0.038056
13	24000.0	791.575	0.016438	0.170271	24775.123692	-0.031786
14	24050.0	751.800	0.016438	0.181933	24775.123692	-0.029705

Fitted polynomial regression models of varying degrees to capture the structure of the implied volatility surface. Employed cross validation techniques and multiple random seeds to enhance the robustness and reliability of the model estimates.

```
[10]: X=vol_data[["maturity","log_moneyness"]]
      y=vol_data["implied_vol"]

      rmse_results = []
      r2_results = []

      for degree in range(1, 10):
          rmse_scores = []
          r2_scores = []
          for seed in range(10,20):
              kf = KFold(n_splits=5, shuffle=True, random_state=seed)
              model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
              rmse_scorer = make_scorer(mean_squared_error, squared=False)
              rmse = cross_val_score(model, X, y, cv=kf, scoring=rmse_scorer).mean()
              r2 = cross_val_score(model, X, y, cv=kf, scoring='r2').mean()
              rmse_scores.append(rmse)
              r2_scores.append(r2)
          avg_rmse = np.mean(rmse_scores)
          avg_r2 = np.mean(r2_scores)
          rmse_results.append(avg_rmse)
          r2_results.append(avg_r2)
      print(f"Degree {degree}: Avg CV RMSE={avg_rmse:.4f}, Avg CV R2={avg_r2:.4f}")
```

```
Degree 1: Avg CV RMSE=0.0081, Avg CV R2=0.3474
Degree 2: Avg CV RMSE=0.0063, Avg CV R2=0.5930
Degree 3: Avg CV RMSE=0.0058, Avg CV R2=0.5574
Degree 4: Avg CV RMSE=0.0067, Avg CV R2=-0.1127
Degree 5: Avg CV RMSE=0.0065, Avg CV R2=0.0958
Degree 6: Avg CV RMSE=0.0143, Avg CV R2=-5.3964
Degree 7: Avg CV RMSE=0.0941, Avg CV R2=-850.7967
Degree 8: Avg CV RMSE=1.7203, Avg CV R2=-237469.5275
Degree 9: Avg CV RMSE=23.5700, Avg CV R2=-32225718.9128
```

```
[ ]:
```

After comparing the RMSE and R-squared values across models, the degree-2 polynomial regression

model was found to provide the best fit. Therefore, the volatility surface will be plotted using this model.

```
[11]: poly = PolynomialFeatures(degree=2, include_bias=False)
      X_poly = poly.fit_transform(X)
      model = LinearRegression()
      model.fit(X_poly, y)
```

```
[11]: LinearRegression()
```

Define ranges for T (time to maturity) and k (log-moneyness) for plotting volatility surface.

```
[12]: T_range = np.linspace(X.iloc[:, 0].min(), X.iloc[:, 0].max(), 50)
      k_range = np.linspace(X.iloc[:, 1].min(), X.iloc[:, 1].max(), 50)
      T_grid, k_grid = np.meshgrid(T_range, k_range)

      X_grid = np.column_stack((T_grid.ravel(), k_grid.ravel()))
      IV_pred = model.predict(poly.fit_transform(X_grid))
      IV_grid = IV_pred.reshape(T_grid.shape)
```

We plot the implied volatility surface using a 3D surface plot with Plotly.

```
[13]: surface = go.Surface(
      x=T_grid, y=k_grid, z=IV_grid,
      colorscale='Viridis',
      opacity=0.9,
      name='Fitted Surface'
    )

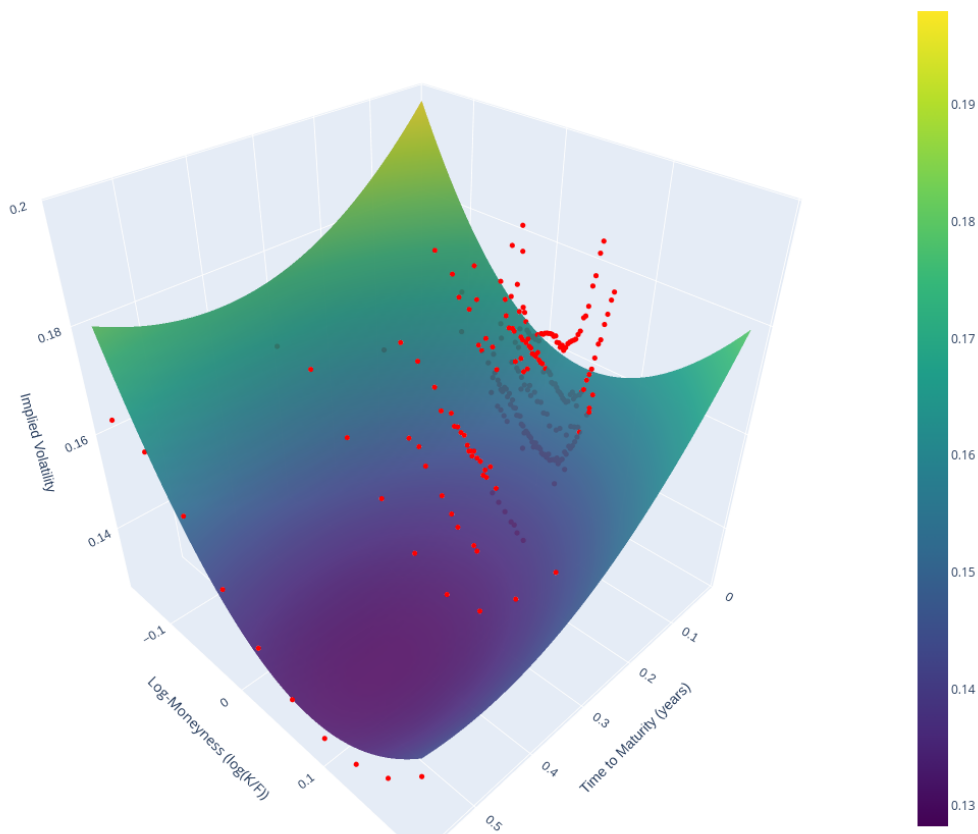
    scatter = go.Scatter3d(
      x=X.iloc[:, 0], y=X.iloc[:, 1], z=y,
      mode='markers',
      marker=dict(size=3, color='red'),
      name='Observed Data'
    )

    layout = go.Layout(
      title="Implied Volatility Surface (Interactive)",
      width=1000,
      height=1000,
      scene=dict(
        xaxis_title="Time to Maturity (years)",
        yaxis_title="Log-Moneyness (log(K/F))",
        zaxis_title="Implied Volatility"
      )
    )
```



```
fig = go.Figure(data=[surface, scatter], layout=layout)
fig.show()
```

Implied Volatility Surface (Interactive)



Residual diagnostic:

```
[14]: print(model.intercept_, model.coef_)
```

```
0.1642695143862547 [-0.14698905 -0.08993028  0.17119076 -0.09322759  0.96146411]
```

The approximate model is:

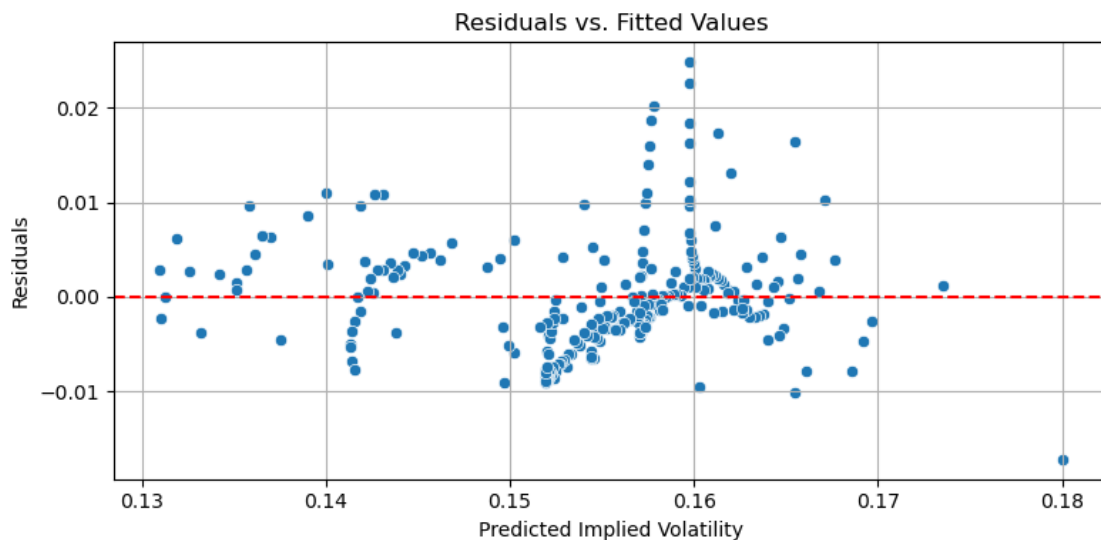
$$\widehat{\sigma(t, k)} = 0.16 - 0.15t - 0.09k + 0.17t^2 - 0.09tk + 0.96k^2$$

```
[15]: y_pred = model.predict(X_poly)
residuals = y - y_pred
```

```
plt.figure(figsize=(8, 4))
sns.scatterplot(x=y_pred, y=residuals)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted Implied Volatility")
plt.ylabel("Residuals")
plt.title("Residuals vs. Fitted Values")
plt.grid(True)
plt.tight_layout()
plt.show()

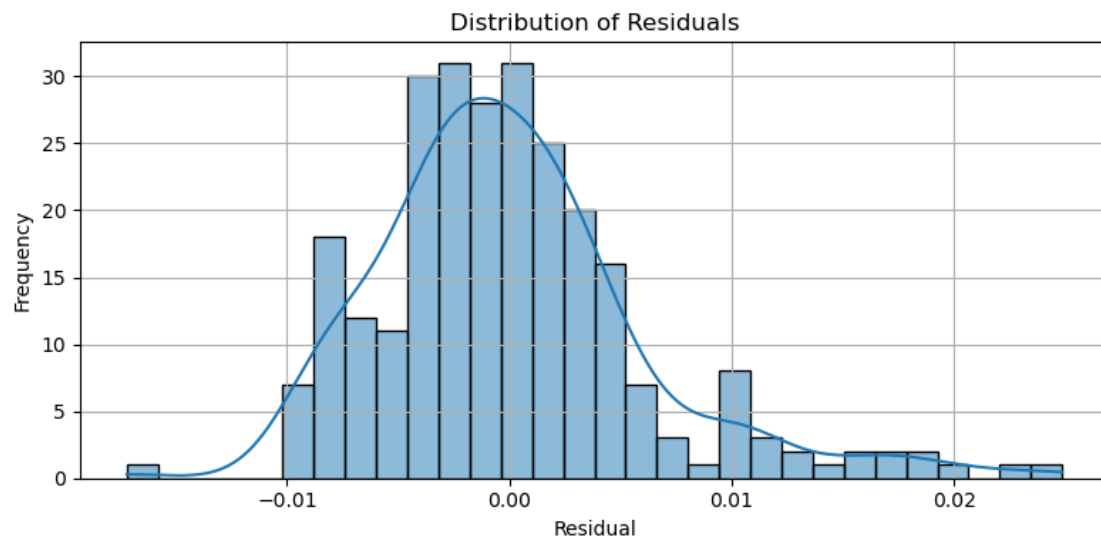
plt.figure(figsize=(8, 4))
sns.histplot(residuals, kde=True, bins=30)
plt.title("Distribution of Residuals")
plt.xlabel("Residual")
plt.ylabel("Frequency")
plt.grid(True)
plt.tight_layout()
plt.show()

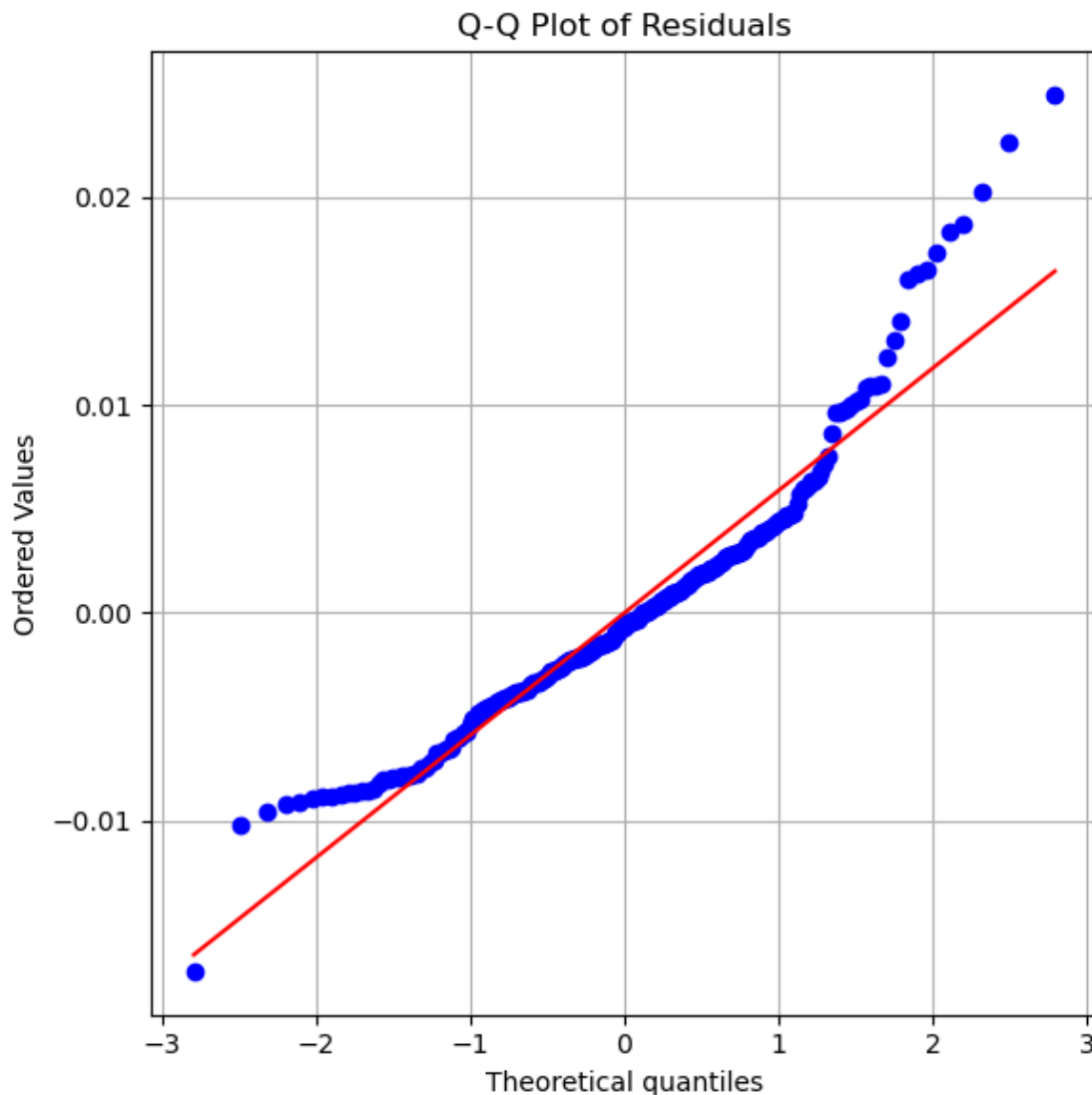
plt.figure(figsize=(6, 6))
stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals")
plt.grid(True)
plt.tight_layout()
plt.show()
```



/home/nirmal/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning:

use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.





A thorough analysis of the residuals from the final model revealed:

- Residuals vs. Fitted: Mostly centered around zero, with minor non-random structure suggesting some curvature not fully captured.
- Histogram: Residuals were approximately symmetric but showed mild skew, likely due to market asymmetries or remaining noise.
- Q-Q Plot: Minor deviations from normality in the tails, indicating that a small number of observations may not conform well to the Gaussian error assumption.
- These diagnostics suggest that while the model explains a significant portion of the variation in implied volatility, some nonlinear or regime-dependent behaviors remain that are not fully captured by a degree-2 polynomial.

Overall, the model provides a reasonably accurate and interpretable representation of the implied

volatility surface for NIFTY options. However, the residual analysis highlights areas for improvement. In future work, incorporating more flexible models (e.g., spline regression, Gaussian processes, or tree-based methods) may better capture complex market structures while maintaining or improving generalization performance.

[]: